Exercise 0

A guided first journey from source code to a running program

Introduction to Computer Science

This exercise is not graded, and you are not expected to know the tools in advance. Its purpose is to let you install everything calmly, try the complete course workflow once, and solve any technical problems before the graded exercises begin. At the end, the submission system will send you an email with a grade of **100**. In this exercise, that number simply means that your setup and submission were successful.

During the exercise, we will move through the same journey that you will follow in later assignments. We will begin with code stored on GitHub, bring a copy to your computer, compile and run it on your local computer and on your Arduino board. Finally, you will connect to the departmental server, compile and run the code there, and submit it. More specifically:

1. You will install Git, VS Code, GCC, and the Arduino command-line tools.
2. You will clone the public Exercise 0 repository to macOS.
3. You will compile and run C code on your local machine.
4. You will compile and upload a program to an Arduino Uno, and then communicate with it through the serial interface.
5. You will connect to the departmental Linux server and test the program there.
6. You will prepare the required ZIP archive, submit it, and understand the feedback returned by the submission system.

1 Before you begin

Before we start, make sure you have your departmental username and password. You can find your username here, under the entry `UNIX-MACS` or `ESC`. Your password is the same password you use to log in to In-Bar.

You will also need a GitHub account. GitHub is the website that we use to distribute the exercise files and keep track of code. If you do not have an account yet, create a free one at github.com. By the way, GitHub offers a Student Developer Pack with various benefits for students. You are welcome to apply for it in your free time (:

We also recommend creating one simple folder for all course work, such as `~/intro2cs`. Keeping the path short and using English letters prevents several surprisingly common problems with compilers and file-transfer tools. For the same reason, avoid placing the course folder inside OneDrive, Google Drive, or another synchronized folder.

2 Install the macOS tools

2.1 Install Git

Git is the version-control tool used to download and manage the exercise repositories. On macOS it is supplied with Apple's Command Line Tools. Open Terminal from Applications > Utilities, or find it with Spotlight, and run:

```
xcode-select --install
git --version
```

If macOS says the tools are already installed, no further action is needed.

2.2 Install an IDE

The official course IDE is Visual Studio Code. You may use another IDE, such as CLion, but the course demonstrations and support material generally assume VS Code, and every exercise must compile with the course GCC commands.

Follow the official VS Code macOS installation guide: download the macOS build, open the `.dmg`, and drag Visual Studio Code into Applications. Open VS Code, install the **C/C++** extension published by Microsoft, then press `Command+Shift+P` and run Shell Command: Install 'code' command in PATH. Open a new Terminal after doing so.

2.3 Install Homebrew and GNU GCC

macOS includes Apple's Clang compiler, which is also invoked by a command named `gcc`. This course uses the GNU GCC compiler, so we install it through Homebrew. Open brew.sh, copy the official installation command shown there, paste it into Terminal, and follow the prompts. Run any Next steps commands printed at the end, then enter:

```
brew update
brew install gcc
ls $(brew --prefix)/bin/gcc-*
```

Homebrew gives GNU GCC a versioned name so that it does not replace Apple's compiler. At the time of writing the command is `gcc-16`; the number on your computer may be newer. In the commands below, replace `gcc-16` with the name printed on your Mac. Verify the tools with:

```
gcc-16 --version
ar --version
```

Checkpoint

Both commands print version information. The first line for `gcc-16` should identify GCC rather than Apple Clang.

2.4 Disable AI assistance

AI tools are becoming an important part of software development, and you will probably use them extensively in the future. During this course, however, we want you to disable them while working on the exercises.

The purpose of an introductory programming course is not only to learn C syntax. More importantly, it is to develop **logical thinking, algorithmic thinking, problem-solving skills, and the ability to debug your own programs**.

AI code generation can be a bit of a **honey pot** at this stage. When you are stuck, it is extremely tempting to let the AI finish a function, fix a bug, or generate an algorithm for you. Unfortunately, the struggle of figuring out these things yourself is exactly where much of the learning happens.

Therefore, please disable VS Code's AI features while working on the course exercises:

1. Open VS Code's Settings by selecting Code > Settings > Settings. You can also press `Command+,`.
2. Enter `chat.disableAIFeatures` in the Settings search box.
3. Find Chat: Disable AI Features and enable its checkbox.

This setting disables and hides VS Code's built-in AI functionality, including AI chat, Copilot, and AI-generated inline suggestions.

You should still keep the **Microsoft C/C++ extension** installed. Its IntelliSense features, such as completing variable and function names and displaying function parameters, are regular programming-editor features and are allowed.

Important

Getting stuck is part of the exercise. Read the error, debug the program, think about the algorithm, ask questions, and try again. These are exactly the skills this course is designed to develop.

3 Clone Exercise 0

The course staff provide every exercise as a public repository under the course GitHub account. For each exercise, you need to clone its repository and work on the local copy on your computer. You are not allowed to push changes to the course repositories, and your account does not have permission to do so.

If you want to keep your work in your own GitHub account, you are invited to copy the repository there. Your personal repository must be **private** so that other students cannot see your solution. Creating this personal copy is optional. Cloning the public course repository itself does not require a GitHub login or authentication token.

Open Terminal and enter:

```
mkdir -p ~/intro2cs
cd ~/intro2cs
git clone https://github.com/intro2cs-biu/ex0.git
cd ex0
ls
```

The `mkdir` command creates a directory, `cd` moves into it, and `git clone` downloads the repository. The final `ls` command lists the files so that you can confirm they arrived.

If command-line Git is unavailable, open github.com/intro2cs-biu/ex0, click Code, choose Download ZIP, extract it, and rename the extracted folder to `ex0`. This alternative does not require Git authentication.

Optionally, you may copy your local clone to your own GitHub account. First install GitHub CLI, then run the following inside the cloned `ex0` folder:

```
gh auth login
gh repo create ex0-private --private --source=. --remote=student --push
```

Follow the browser authentication flow. Before continuing, open the new repository on GitHub and verify that it is marked **Private**.

Important

Making exercise solutions publicly accessible is strictly prohibited. A public solution can be copied by others and may lead to an academic-integrity review for the students involved. To protect both yourself and your classmates, keep any personal repository private and do not share access to it.

Checkpoint

The `ls` command shows `ex0-desktop.c` and `ex0-arduino.c`. If you used `git clone`, `git status` also says you are inside a working tree.

4 Compile and run the first C program

You now have your first C source file. A source file is still just text, the computer cannot execute it directly. We will open it in VS Code, ask GCC to compile it, and then run the executable that GCC creates.

Open the cloned folder in VS Code by entering:

```
code .
```

The dot means "the current folder." When VS Code opens, select `ex0-desktop.c`. You will see a short program whose job is to print `hello world`. Return to Terminal, make sure you are still in the folder containing `ex0-desktop.c`, and compile it with:

```
gcc-16 -std=c99 -Wall -Wextra ex0-desktop.c -o ex0-desktop
```

Here, `-std=c99` asks GCC to use the version of C required by the course, and `-Wall -Wextra` asks it to warn us about issues in the code. The `-o ex0-desktop` part names the program that GCC creates. If the command ends without an error message, compilation succeeded. Now run the new program:

```
./ex0-desktop
```

The Terminal should display:

```
hello world
```

Tip

Compilation and execution are different steps. GCC creates `ex0`; typing `./ex0-desktop` runs the program from the current directory. Recompile after every source change.

5 Install the Arduino command-line tools

5.1 Install Arduino CLI

Arduino CLI is the official command-line tool for working with Arduino boards. We will use it to detect connected boards, compile and upload programs, and communicate with the board through the serial monitor.

Install Arduino CLI with Homebrew, then install support for the Arduino Uno:

```
brew install arduino-cli
arduino-cli version
arduino-cli core update-index
arduino-cli core install arduino:avr
arduino-cli core list
```

The final command should list `arduino:avr`. This also installs Arduino's board support and drivers/tools used for an Uno.

5.2 Install the raw AVR compiler and uploader

The processor inside an Arduino Uno is different from the processor inside your computer. Consequently, the normal macOS compiler cannot create a program for the Uno. We use `avr-gcc`, a version of GCC that targets AVR microcontrollers. The program `avrdude` then transfers the compiled result to the board.

Install the AVR compiler from the maintained Homebrew AVR tap, and install the uploader from Homebrew:

```
brew tap osx-cross/avr
brew install avr-gcc avrdude
```

Then open a new Terminal and verify:

```
avr-gcc --version
avr-objcopy --version
avrdude --version
```

6 Download and install the Intro2CS library

The Arduino tools know how to compile a program for the Uno, but they do not yet provide the functions used to control the components in the course kit. The Intro2CS library adds these functions.

The library has an official course tutorial and documentation. It contains the current library download, wiring diagrams, compilation commands, and documentation for each library module.

Use the guide to install the library as follows:

1. Download intro2cs-lib-macos.zip library from this link.
2. Move the downloaded ZIP file into your main course folder, `~/intro2cs`. This is the folder that already contains the cloned `ex0` repository.
3. Double-click the ZIP in Finder to extract it. The folder named `intro2cs-lib` should be beside `ex0`, not inside it.

Your main course folder should now look like this:

```
~/intro2cs/
|-- intro2cs-lib/
|   |-- headers/
|   |-- intro2cs-arduino-lib.a
|   `-- intro2cs-arduino-simulator-lib.a
|-- ex0/
|   |-- ex0-desktop.c
|   `-- ex0-arduino.c
|-- ex1/
|-- ...
```

The `ex1` line illustrates where later repositories will go, it does not need to exist yet. If you see another `intro2cs-lib` folder inside the first one, move the inner folder up one level.

In future exercises, clone each repository into `~/intro2cs`. The relative path `../intro2cs-lib` will then continue to work without installing the library again.

7 Connect the Arduino circuit

We are now ready to run our first piece of code on the Arduino board. Make sure the usb cable is disconnected from the board before adding or moving wires. This protects the board from an accidental short circuit while you work. Build the circuit shown below using the Arduino Uno, breadboard, LED with its series resistor, and the 16-by-2 LCD. Take your time and follow one wire at a time. Before reconnecting USB, check the LED polarity and verify each power and ground connection.

Before compiling, open `ex0-arduino.c` in VS Code and take a moment to read through it. You are not expected to understand every detail yet, we will study these ideas more carefully during the course. For now, let us briefly see what each line asks the Arduino to do:

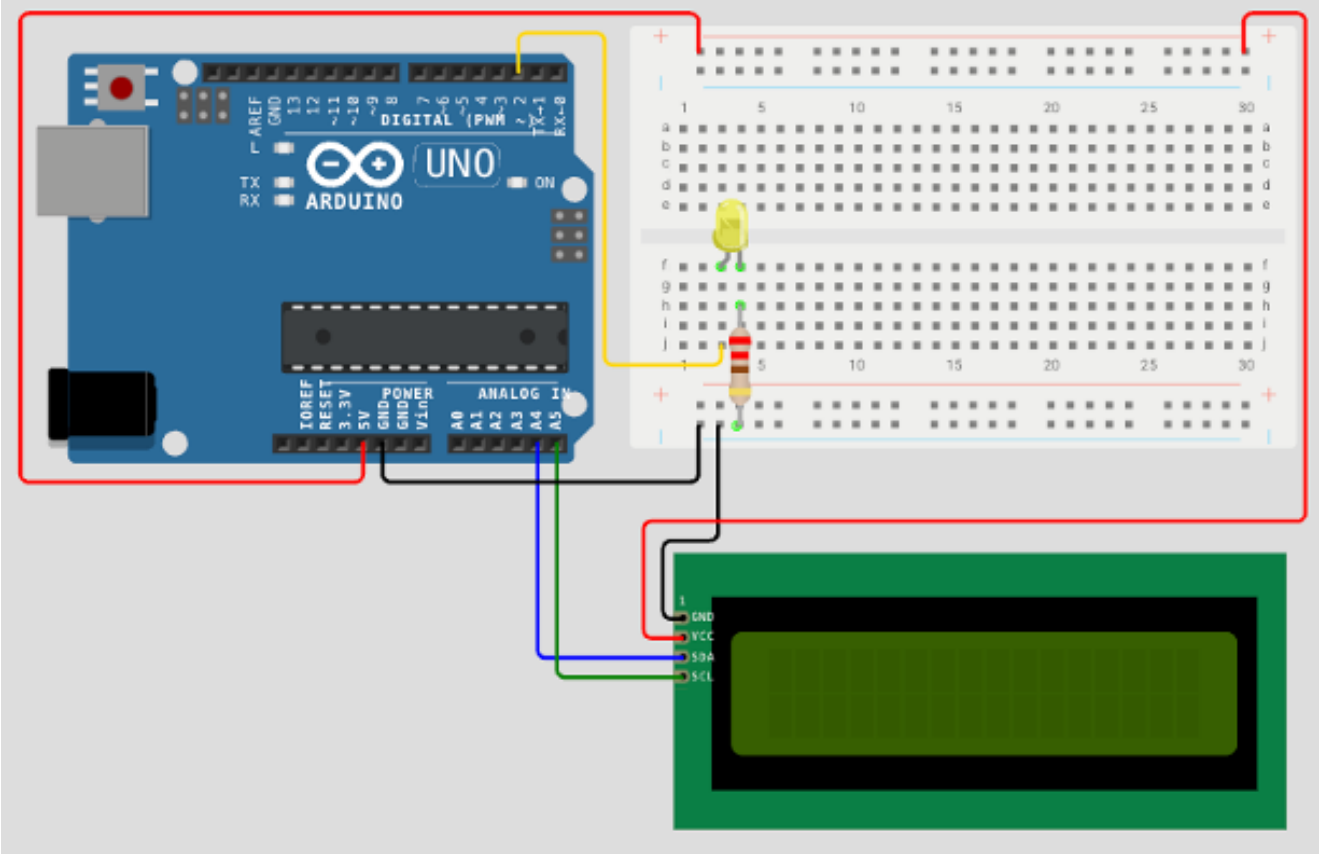


Figure 1: Exercise 0 circuit. Follow the diagram exactly.

```
#include "arduino.h"
#include "lcd.h"
#include "serial.h"
#include <stdio.h>

int main(void) {
    pinMode(2, OUTPUT);
    lcdBegin(16, 2);
    serialBegin(9600);
    printf("Hello, World!\n");
    lcdPrintf("Hello, World!");
    digitalWrite(2, HIGH);
}
```

The `#include` lines make functions from the course library and the standard C library available to the program. The function `main` is where the program begins running. Inside it, `pinMode(2, OUTPUT)` prepares pin 2 to send an electrical signal, and `lcdBegin(16, 2)` prepares the LCD, which has 16 characters in each of its two rows.

The call `serialBegin(9600)` starts communication with the computer at a speed of 9600 baud. The following `printf` sends `Hello, World!` to the computer through that serial connection, while `lcdPrintf` displays the same message on the LCD. Finally, `digitalWrite(2, HIGH)` sends a HIGH signal through

pin 2, which turns on the connected LED.

This quick explanation is enough for now. You will learn what functions, headers, pins, serial communication, and the values `HIGH` and `OUTPUT` mean in much greater detail as the course progresses.

8 Compile and upload to the Arduino Uno

Compiling for the Arduino has two visible stages. First, `avr-gcc` produces an ELF file containing the machine code and useful metadata. Then `avr-objcopy` creates the smaller HEX file that will be uploaded to the board. Run the following commands:

```
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os -std=c99 -I../intro2cs-lib/headers ex0-arduino.
    c ../intro2cs-lib/intro2cs-arduino-lib.a -o ex0-arduino.elf

avr-objcopy -O ihex -R .eeprom ex0-arduino.elf ex0-arduino.hex
```

If your files are stored in different folders, update the paths in the commands accordingly. In future exercises, you will also need to replace the exercise file names (`ex0-arduino.c`, `ex0-arduino.elf`, and `ex0-arduino.hex`) with the appropriate names for that exercise.

Now connect the Arduino board to your computer using the USB cable. macOS assigns the connected board a serial-port name such as `/dev/cu.usbmodem2101`. The number can change when you use a different USB socket, so we should ask Arduino CLI instead of guessing. Run the following command:

```
arduino-cli board list
```

Look for `Arduino Uno`, FQBN `arduino:avr:uno`, and a port such as `/dev/cu.usbmodem2101`. The FQBN is Arduino's precise name for the type of board. To upload the code to the board, replace `/dev/cu.usbmodem2101` in the following command with the port shown on your computer:

```
avrdude -p atmega328p -c arduino -P /dev/cu.usbmodem2101 -b 115200 -D -U flash:w:ex0-arduino.
    hex:i
```

Checkpoint

If everything worked correctly, the LCD should display `Hello, World!`, and the LED should be lit.

9 Serial interface tutorial

So far, we have used the computer only to upload a program to the Arduino. Serial communication creates an ongoing connection between the computer and the board, allowing them to exchange information while the program is running. The Arduino board can send text and values to the computer, and the computer can send input back to the Arduino. This makes it especially useful for observing, interacting with, and debugging a running program.

The communication travels through the USB cable. Both sides must agree on a speed, called the *baud rate*, in this exercise that speed is 9600. The starter program contains the following essential pattern:

```
#include "serial.h"
#include <stdio.h>
```

```
int main(void) {
    serialBegin(9600);
    printf("Hello, World!\n");
}
```

The header makes the course serial function available. Calling `serialBegin(9600)` connects the familiar standard C input and output functions to the Arduino serial port. Always call it once before using `printf`, `getchar`, or `scanf`. Ending output with `\n` starts a new line, which makes the terminal readable.

9.1 Open the serial monitor

A serial monitor is a small terminal window connected to the board. First ask Arduino CLI for the current port, then open that port at 9600 baud:

```
arduino-cli board list
arduino-cli monitor -p /dev/cu.usbmodem2101 --config baudrate=9600
```

Remember to replace `/dev/cu.usbmodem2101` with your port. You should see `Hello, World!`. Press `Ctrl+C` when you want to close the monitor. If Arduino CLI reports a configuration error, ask it to describe the settings supported by this port:

```
arduino-cli monitor -p /dev/cu.usbmodem2101 --describe
```

Press the Uno's reset button after opening the monitor if the startup message has already passed. Opening a serial connection can itself reset many Uno boards, so a repeated startup message is normal.

Important

`/dev/cu.usbmodem2101` is only an example. Close any serial monitor before re-uploading your code; only one program can normally use the port at a time.

10 Test on the departmental server

The departmental server is a Linux computer maintained by the department. Running your program there confirms that it behaves correctly in the same environment used by the course systems and staff.

Before your first connection, you must set up two-factor authentication:

1. Download and install **Google Authenticator** on your phone (iPhone, Android).
2. Go to mfar.esc.biu.ac.il and enter your username.
3. After confirming your username, a QR code will be sent to your Bar-Ilan email address. Open Google Authenticator and scan this QR code.
4. Google Authenticator will now display a six-digit verification code that changes every 30 seconds. You will use this code when connecting to the departmental server.

Open Terminal and connect through the departmental gateway. Replace both occurrences of `username` with your departmental username:

```
ssh -J username@vpn.esc.biu.ac.il:2222 username@planet.cs.biu.ac.il
```

After connecting, create `~/intro2cs/ex0`, type `exit`, and copy both source files through the same gateway.

The departmental server needs the complete **Linux library package**, including the headers, the Arduino library, and the simulator library. Prepare a separate copy for the server as follows:

1. On your computer, download `intro2cs-lib-linux.zip`.
2. Inside your local course folder, create a folder named `server-linux`. Extract the Linux ZIP into it. Open `server-linux` and check that it contains a folder named `intro2cs-lib`, with the files shown below. Keep the library you already use locally in its original location.

Your **local computer** should now have this structure:

```
~/intro2cs/
|-- ex0/
|   |-- ex0-desktop.c
|   `-- ex0-arduino.c
|-- intro2cs-lib/          (your local OS package)
`-- server-linux/
    `-- intro2cs-lib/      (extracted from the Linux ZIP)
        |-- headers/
        |-- intro2cs-arduino-lib.a
        `-- intro2cs-arduino-simulator-lib.a
```

3. In your local Terminal, open `~/intro2cs/ex0`. Copy the source files and the Linux library folder through the gateway:

```
cd ~/intro2cs/ex0
scp -J username@vpn.esc.biu.ac.il:2222 ex0-desktop.c ex0-arduino.c username@planet.cs.biu
    .ac.il:~/intro2cs/ex0/
scp -J username@vpn.esc.biu.ac.il:2222 -r ../server-linux/intro2cs-lib username@planet.cs
    .biu.ac.il:~/intro2cs/
```

Check that the **departmental server** has this structure:

```
~/intro2cs/
|-- ex0/
|   |-- ex0-desktop.c
|   `-- ex0-arduino.c
`-- intro2cs-lib/          (the Linux package)
    |-- headers/
    |-- intro2cs-arduino-lib.a
    `-- intro2cs-arduino-simulator-lib.a
```

On the server, `intro2cs-lib` belongs directly inside `~/intro2cs`, beside `ex0`. Upload the folder from `server-linux`, not the library used locally on your computer.

Connect again, then compile and run the program:

```
cd ~/intro2cs/ex0
gcc -std=c99 -Wall -Wextra ex0-desktop.c -o ex0-desktop
./ex0-desktop
```

The expected output is `hello world`.

10.1 Verify the Arduino compilation on the server

You can also use the departmental server to check that your Arduino source code compiles correctly. This is useful because it tests the build in a standard course environment before you upload the result to the physical board.

Arrange the departmental server like your computer: under `~/intro2cs`, keep `ex0` and `intro2cs-lib` as neighboring folders, not one inside the other. The preceding copy commands create this structure using the Linux library package; confirm it before compiling.

In the remote server terminal, enter:

```
cd ~/intro2cs/ex0
avr-gcc --version
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os -std=c99 -I../intro2cs-lib/headers ex0-arduino.
    c ../intro2cs-lib/intro2cs-arduino-lib.a -o ex0-arduino.elf
avr-objcopy -O ihex -R .eeprom ex0-arduino.elf ex0-arduino.hex
```

If these commands finish without an error, the Arduino program compiled successfully. The file that must be uploaded to the board is `ex0-arduino.hex`. Leave the server and copy it back to the current local folder:

```
scp -J username@vpn.esc.biu.ac.il:2222 username@planet.cs.biu.ac.il:~/intro2cs/ex0/ex0-arduino.
    hex .
```

The Arduino is connected by USB to your Mac, not to the remote departmental server. For that reason, the remote server normally cannot upload directly to the board through SSH. After downloading the HEX file, close the serial monitor and upload it from a local Terminal, replacing `/dev/cu.usbmodem2101` with the port shown on your computer:

```
arduino-cli board list
avrdude -p atmega328p -c arduino -P /dev/cu.usbmodem2101 -b 115200 -D -U flash:w:ex0-arduino.
    hex:i
```

10.2 Getting More Comfortable with Linux

If you would like to become more comfortable with the Linux environment and the commands used above, we recommend working through this introductory Linux tutorial. You do not need to complete the entire tutorial for Exercise 0, but its sections about navigating directories, listing files, and working with the command line are useful throughout the course.

11 Compare your output with the submission simulator

When you submit an exercise, the submission system runs basic automatic tests. These tests do not connect your program to a physical Arduino. Instead, they use the course simulator library, which

replaces hardware actions with text. For example, an instruction that sets a pin to HIGH produces a line of text describing that action. The tests compare the program's output with the output expected for the exercise.

This is the reason you may want to run the simulator on your own computer and on the departmental server: it allows you to see exactly what the basic automatic tests will see. It is not a replacement for using the Arduino board. We use it only to compare textual output before submitting.

Compile the same `ex0-arduino.c` source file with the Homebrew GNU `gcc-16` compiler, but link it with `intro2cs-arduino-simulator-lib.a` instead of `intro2cs-arduino-lib.a`. The simulator library implements the same course functions while printing their effects in the terminal.

On your Mac, open Terminal in `~/intro2cs/ex0` and enter:

```
gcc-16 -std=c99 -Wall -Wextra -I../intro2cs-lib/headers ex0-arduino.c ../intro2cs-lib/intro2cs
    -arduino-simulator-lib.a -o ex0-arduino

./ex0-arduino
```

On the departmental server, use the Linux package copied earlier and the server's `gcc` compiler. In the remote terminal, enter:

```
cd ~/intro2cs/ex0
gcc -std=c99 -Wall -Wextra -I../intro2cs-lib/headers ex0-arduino.c ../intro2cs-lib/intro2cs-
    arduino-simulator-lib.a -o ex0-arduino
./ex0-arduino
```

Compare these commands with the physical-Arduino build used earlier. The physical build uses `avr-gcc` and the Arduino archive. The simulator build uses the computer's C compiler and the simulator archive. Your C source and header includes remain unchanged.

For the starter program, the simulator output contains `Hello, World!`, the rendered LCD contents, and an indication that pin 2 was set to HIGH. Compare the order, spelling, capitalization, spaces, and numeric values carefully. Automated comparisons are exact, so output that looks equivalent to a person may still differ because of one extra character or a missing newline.

Important

Passing this comparison only shows that the program produced the expected basic simulator output. You must still compile and run the program on the real Arduino.

12 Prepare the ZIP submission

1. Open the cloned `ex0` folder in Finder.
2. Select the required repository contents, including `ex0-arduino.c` and `ex0-desktop.c`. Do not include generated executables, `.elf`, `.hex`, `.o`, `intro2cs-lib`, library archives, or simulator files. Those files are needed for compilation but must not be submitted.
3. Control-click the selected files and choose Compress Items. Finder creates a ZIP archive in the same folder. Rename it `ex0.zip` and open it once to verify its contents.

13 Submit and verify the result

You have now reached the final step. Open submit.cs.biu.ac.il. Sign in with your departmental username, password, and verification code. Choose course **89-110**, group **01**, and Exercise **0**, choose the ZIP, send it, and confirm.

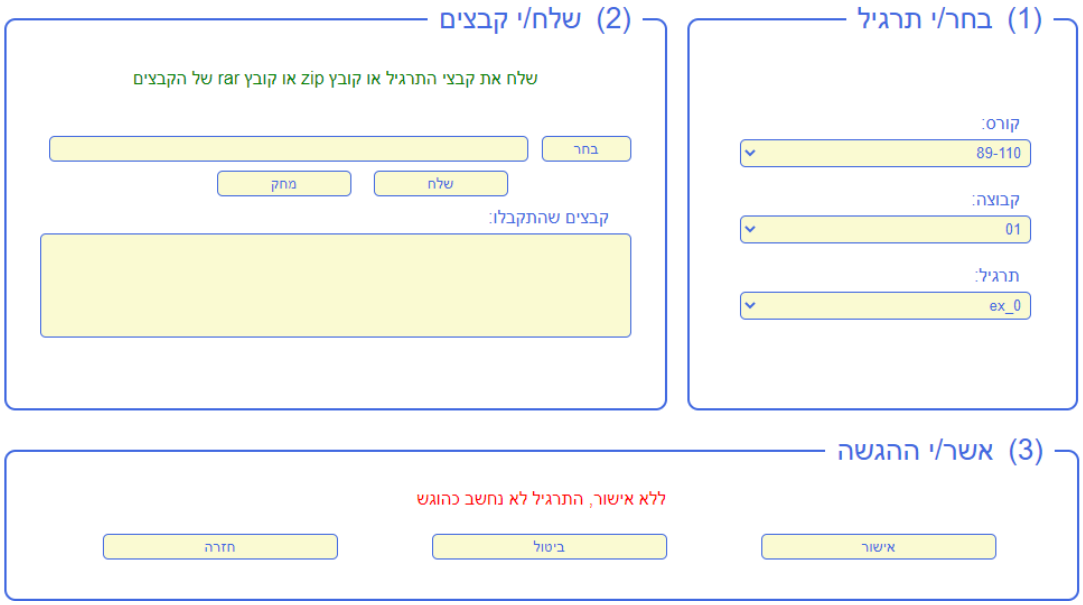


Figure 2: Select the course, group, exercise, and ZIP before confirming.

After submission, the system examines the archive and sends feedback, normally to your university email address. A successful report ends with a grade of 100.

Checkpoint

Do not consider Exercise 0 complete until the feedback email shows **100**. If the grade is lower, read the report, correct the named issue, rebuild the ZIP, and submit again.

14 If something does not work

Technical setup problems are normal, especially the first time several tools must work together. An error message is information, not a sign that you are bad at programming. Read the first relevant error line, find the matching case below, and check one possible cause at a time.

`gcc` or `git` is not recognized
Open a new Terminal after installing Homebrew or the Command Line Tools. Run `brew -prefix` and `ls $(brew -prefix)/bin/gcc-*` to find the versioned GNU GCC command.

Header file such as `arduino.h` is missing
Run the command from `ex0` and verify that `..\intro2cs-lib\headers` exists. Do not change include lines to absolute paths.

No board appears
Try another data-capable USB cable or port, reconnect the board, and rerun `arduino-cli board list`. Also check Apple menu > About This Mac > System Report > USB.

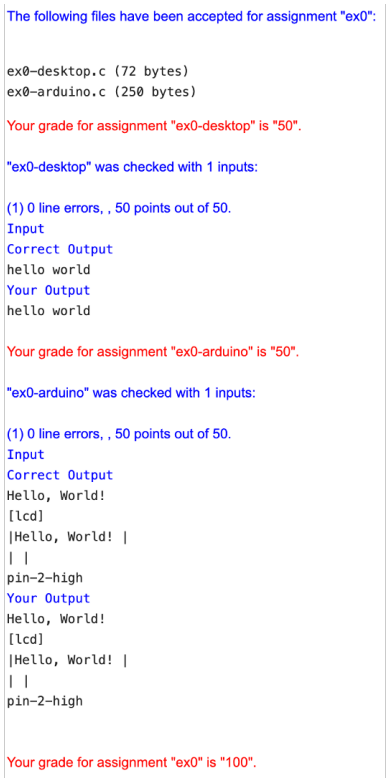


Figure 3: Example of successful Exercise 0 feedback.

Upload cannot open the serial port
Close the serial monitor and any Arduino IDE window, confirm the port again, and retry. Replace the example `/dev/cu.usbmodem2101` path everywhere.

No serial text
Set the monitor to 9600 baud, open it, and press Reset. Confirm that the code calls `serialBegin(9600)` before printing.

Support resources: faculty support, submission troubleshooting, and helpdesk. Consult classmates and the course forum before opening a helpdesk request.

15 Before you finish

Take one final minute to review what you accomplished. Every statement below should now be true. If one is not, return to that section before moving on to a graded exercise.

- ☐ I can open Git, GCC, and VS Code on my Mac.
- ☐ I cloned Exercise 0, and if I created a personal GitHub copy, I made sure that I am **not** storing solutions in a public repository.
- ☐ I compiled `ex0-desktop.c` locally and saw it print `hello world`.
- ☐ I ran the program on the real Arduino and saw its text in the serial monitor.
- ☐ I compared the program's textual output with the output produced by the submission simulator.

☐ I compiled and ran `ex0-desktop.c` on `planet.cs.biu.ac.il`.

☐ I opened the ZIP and confirmed that `ex0-desktop.c` and `ex0-arduino.c` are both at its top level.

☐ I received a submission feedback email showing **100**.